

Cahier des Charges

Titre du projet : Mise en place d'une Infrastructure as Code Cloud-Ready en environnement local pour un SIRH avec auto-scaling et CI/CD

Étudiant : Eya MHIR

Entreprise d'accueil : Sopra HR Software

Tuteur entreprise : Ramzi BEN CHAMEKH

Tuteur académique : Fatma LOUATI

Durée du stage : 6 mois (janvier- Juillet 2026)

Domaine : Cloud & DevOps

Date de création : Janvier 2026

Titre du projet

Mise en place d'une Infrastructure as Code Cloud-Ready en environnement local pour un SIRH avec auto-scaling et CI/CD

1. Introduction

Dans un contexte de transformation digitale, les Systèmes d'Information des Ressources Humaines (SIRH) jouent un rôle critique dans la gestion des collaborateurs (congés, paie, recrutements, performances). Ces systèmes doivent être hautement disponibles, scalables et sécurisés, notamment lors de pics d'activité.

Ce projet est réalisé au sein de Sopra HR Software, éditeur européen leader dans le domaine des solutions SIRH. L'entreprise accompagne ses clients dans la modernisation de leurs systèmes RH, ce qui rend pertinent l'exploration d'architectures Cloud-Ready pour ses environnements de développement et de test.

Cependant, dans certains environnements industriels, l'accès aux Clouds publics (AWS, Azure, GCP) peut être restreint pour des raisons de sécurité, de coûts ou de politiques internes. Il devient alors essentiel de former et concevoir des architectures Cloud-Ready en environnement local, tout en respectant les bonnes pratiques DevOps et IaC.

Ce projet s'inscrit dans ce cadre et vise à :

- Simuler une infrastructure Cloud complète en local,
- Automatiser son déploiement via l'Infrastructure as Code,

- Héberger une application SIRH simulée,
- Implémenter des mécanismes d'auto-scaling,
- Industrialiser le cycle de vie de l'infrastructure.

2. Problématique

Les infrastructures traditionnelles déployées manuellement présentent plusieurs limites :

- Faible reproductibilité des environnements
- Risque élevé d'erreurs humaines
- Difficulté de montée en charge
- Manque de visibilité et d'automatisation
- Absence d'industrialisation des déploiements

Problématique principale :

Comment concevoir et automatiser une infrastructure Cloud-Ready pour un SIRH, entièrement déployée en environnement local, tout en garantissant la scalabilité, la disponibilité et l'industrialisation des déploiements grâce à l'Infrastructure as Code et aux pratiques DevOps ?

3. Objectifs du projet

3.1 Objectif général

Concevoir et mettre en œuvre une infrastructure Cloud-Ready automatisée en environnement local, destinée à héberger un SIRH simulé, intégrant des mécanismes d'auto-scaling, de monitoring et de CI/CD.

3.2 Objectifs spécifiques

- Concevoir une architecture Cloud-Ready simulée en local
- Automatiser l'infrastructure via **Terraform**
- Automatiser la configuration via **Ansible**
- Déployer un **cluster Kubernetes local**
- Mettre en place l'auto-scaling horizontal
- Déployer une **application SIRH simulée**
- Développer un **dashboard React**

- Mettre en place un pipeline CI/CD IaC
- Appliquer des bonnes pratiques DevOps & sécurité
- Documenter entièrement la solution

4. Besoins fonctionnels et non fonctionnels

4.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit fournir pour répondre aux objectifs du projet.

4.1.1 Infrastructure as Code (IaC)

- **Provisioning automatisé** : Le système doit permettre le déploiement automatique de l'infrastructure complète via Terraform (réseaux, cluster, namespaces) en une seule commande.
- **Configuration automatisée** : Ansible doit automatiser la configuration des environnements (installation des dépendances, sécurisation, paramétrage Kubernetes).
- **Modularité IaC** : Les modules Terraform doivent être indépendants, réutilisables et versionnés (module réseau, module cluster, module namespaces).
- **Multi-environnements** : Le système doit gérer trois environnements isolés (DEV, TEST, PROD-SIM) via des namespaces Kubernetes distincts

4.1.2 Auto-scaling

- **Scaling horizontal automatique** : Le HPA (Horizontal Pod Autoscaler) doit ajuster automatiquement le nombre de réplicas de l'API en fonction de la charge CPU/mémoire.
- **Seuils de scaling configurables** : Les seuils de déclenchement de l'auto-scaling (CPU, mémoire) doivent être paramétrables par environnement.

4.1.3 Monitoring & Observabilité

- **Collecte de métriques** : Prometheus doit collecter les métriques des pods, des nœuds et des services applicatifs (CPU, mémoire, requêtes HTTP).
- **Visualisation des métriques** : Grafana doit fournir des dashboards de visualisation en temps réel de l'état du cluster et des applications.
- **Alerting** : Des alertes doivent être déclenchées en cas de surcharge CPU, de redémarrages anormaux (CrashLoopBackOff) ou d'indisponibilité de l'API.

4.1.4 CI/CD & Industrialisation

- **Pipeline CI/CD automatisé** : Un pipeline GitLab CI doit automatiser la validation, le linting, le scan de sécurité et le déploiement de l'infrastructure.

- **Validation avant déploiement** : Chaque modification de code IaC doit passer par les étapes terraform fmt, terraform validate, tfint et checkov avant tout déploiement.
- **Déploiement progressif** : Le pipeline doit déployer séquentiellement sur DEV → TEST → PROD-SIM, avec une validation manuelle requise avant le déploiement en PROD-SIM.
- **Tests automatisés** : Des tests d'infrastructure et des smoke tests applicatifs doivent être exécutés automatiquement après chaque déploiement

4.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité, de performance et les contraintes techniques du système.

4.2.1 Performance

- **Temps de déploiement** : L'infrastructure complète doit être déployée en moins de 15 minutes via Terraform et Ansible.
- **Réactivité de l'auto-scaling** : Le HPA doit déclencher le scaling en moins de 2 minutes après le dépassement du seuil CPU configuré.

4.2.2 Disponibilité & Résilience

- **Disponibilité de l'API** : L'API doit être disponible à 99% dans l'environnement PROD-SIM (hors maintenance planifiée).
- **Redémarrage automatique** : Les pods en erreur doivent être automatiquement redémarrés par Kubernetes (politique de restart).
- **Tolérance aux pannes** : La perte d'un pod API ne doit pas entraîner d'interruption de service (grâce aux réplicas multiples en PROD-SIM).

4.2.3 Sécurité

- **Isolation des environnements** : Les namespaces DEV, TEST et PROD-SIM doivent être isolés (pas de communication directe inter-namespaces).
- **Gestion des secrets** : Aucun secret (mot de passe, clé) ne doit être stocké en clair dans le code source ; utilisation obligatoire de Kubernetes Secrets.
- **Conformité IaC** : 100% du code Terraform doit passer les scans de sécurité Checkov sans vulnérabilité critique.
- **Contrôle d'accès** : Le RBAC Kubernetes doit être activé pour restreindre les accès selon les rôles.

4.2.4 Reproductibilité & Maintenabilité

- **Reproductibilité** : L'infrastructure doit être entièrement reproductible : un terraform destroy suivi d'un terraform apply doit restaurer un état identique.

- **Idempotence** : Les playbooks Ansible et les modules Terraform doivent être idempotents (exécution multiples = même résultat).
- **Documentation** : Chaque module Terraform et playbook Ansible doit être documenté (README, variables, exemples d'utilisation).
- **Versionning** : Tout le code IaC doit être versionné sous Git avec un historique de commits clair et des messages descriptifs.

4.2.5 Portabilité & Évolutivité

- **Portabilité Cloud-Ready** : L'architecture doit être conçue pour être transposable vers un Cloud public (AWS, Azure, GCP) avec des modifications minimales des modules Terraform.
- **Évolutivité** : L'ajout d'un nouveau microservice ou d'un nouvel environnement doit être réalisable en ajoutant un module Terraform sans modifier l'existant.

5. Périmètre du projet

5.1 Inclus

- Infrastructure as Code (Terraform: modules versionnés)
- Provisioning et configuration (Ansible : playbooks automatisés)
- Kubernetes local (Minikube: orchestration des services)
- Application SIRH simulée :
 - ✓ API RH (microservice backend)
 - ✓ Base de données (PostgreSQL)
 - ✓ Dashboard React
- Auto-scaling (Horizontal Pod Autoscaler (HPA))
- Monitoring (Prometheus & Grafana)
- Pipeline CI/CD IaC : intégration et déploiement continu de l'IaC
- Documentation technique et utilisateur

5.2 Hors périmètre

- Déploiement réel sur Cloud public (AWS, Azure, GCP)
- Développement d'une application SIRH complète avec logique métier avancée

6. Architecture générale

6.1 Vue globale

L'architecture repose sur une simulation Cloud locale, inspirée des architectures Cloud industrielles.

Couches principales :

1. Infrastructure
2. Configuration
3. Application SIRH
4. Monitoring & Observabilité
5. CI/CD & Industrialisation

6.2 Architecture technique (logique)

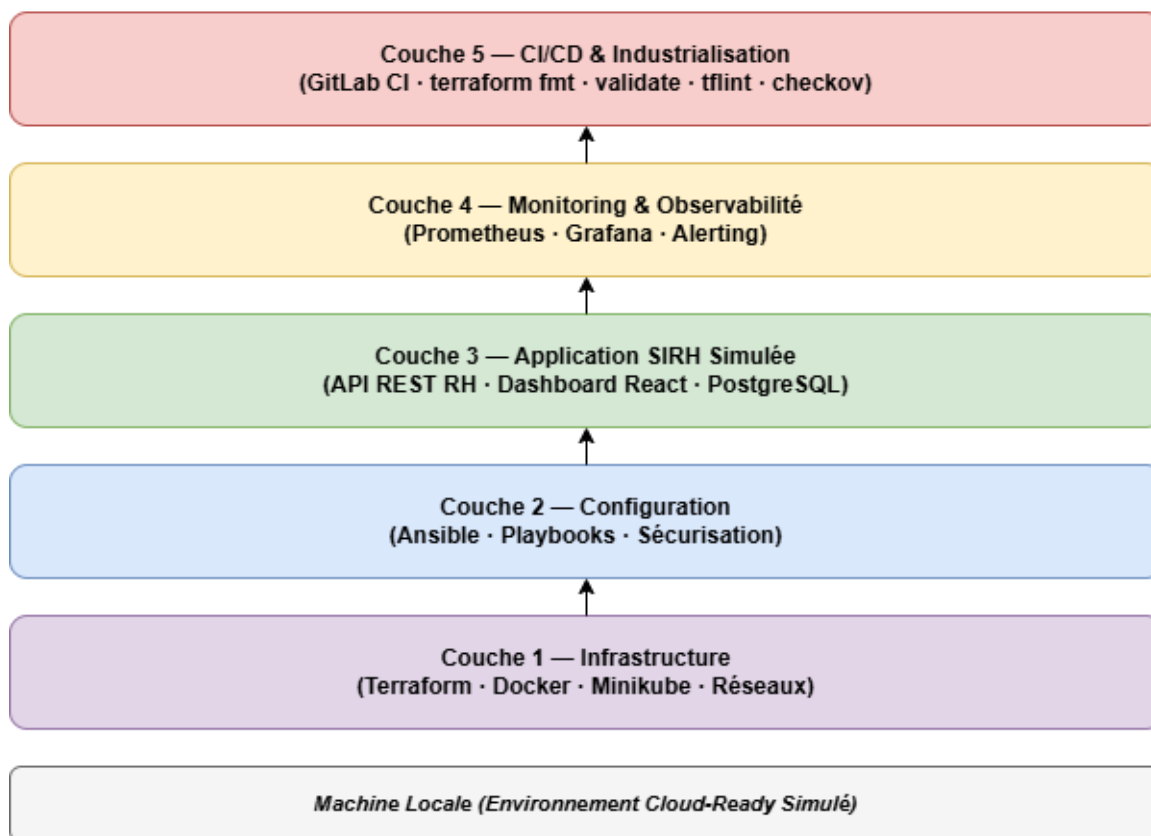
- **Terraform**: gestion de l'infrastructure (modules, réseau, cluster)
- **Ansible** : configuration et sécurisation
- **Kubernetes (Minikube)** : orchestration des services
- **Monitoring** : Prometheus (collecte), Grafana (visualisation), alerting

6.3 Diagrammes d'architecture

Les diagrammes suivants illustrent les différentes vues de l'architecture du projet, depuis la vision d'ensemble jusqu'au détail du déploiement et de l'industrialisation.

- **Architecture globale** :

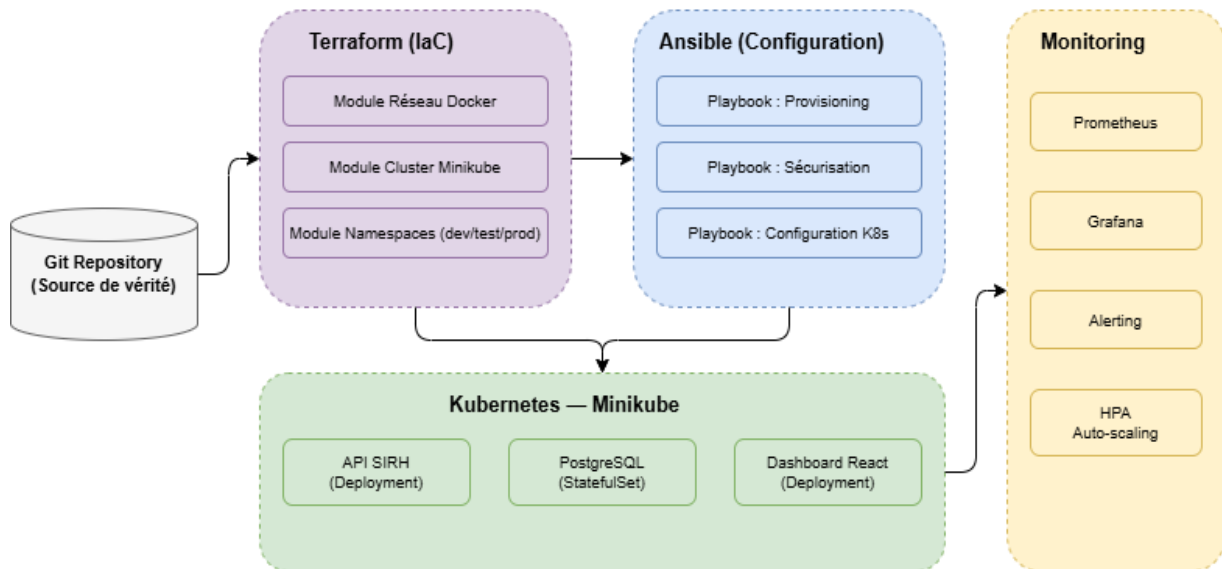
Ce diagramme présente une vue d'ensemble de l'architecture du projet, organisée en cinq couches superposées. Chaque couche représente un niveau de responsabilité distinct, allant de l'infrastructure de base (Terraform, Docker, Minikube) jusqu'à l'industrialisation via le pipeline CI/CD. Cette représentation en couches permet de visualiser comment les différents composants s'empilent et interagissent entre eux pour former une infrastructure Cloud-Ready cohérente, entièrement déployée en environnement local.

Figure 1 — Architecture Globale du Projet

- **Architecture logique (par couches + responsabilités)**

Ce diagramme détaille l'architecture logique du projet en mettant en évidence les responsabilités de chaque composant technique. On y distingue le rôle de Terraform dans la gestion de l'infrastructure (modules réseau, cluster, namespaces), celui d'Ansible dans l'automatisation de la configuration et de la sécurisation, ainsi que l'orchestration des services applicatifs sur Kubernetes (Minikube). La couche Monitoring (Prometheus, Grafana, AlertManager, HPA) assure l'observabilité et l'auto-scaling. Le tout est piloté par Git comme source de vérité unique, selon une approche GitOps.

Figure 2 — Architecture Logique (Couches & Responsabilités)



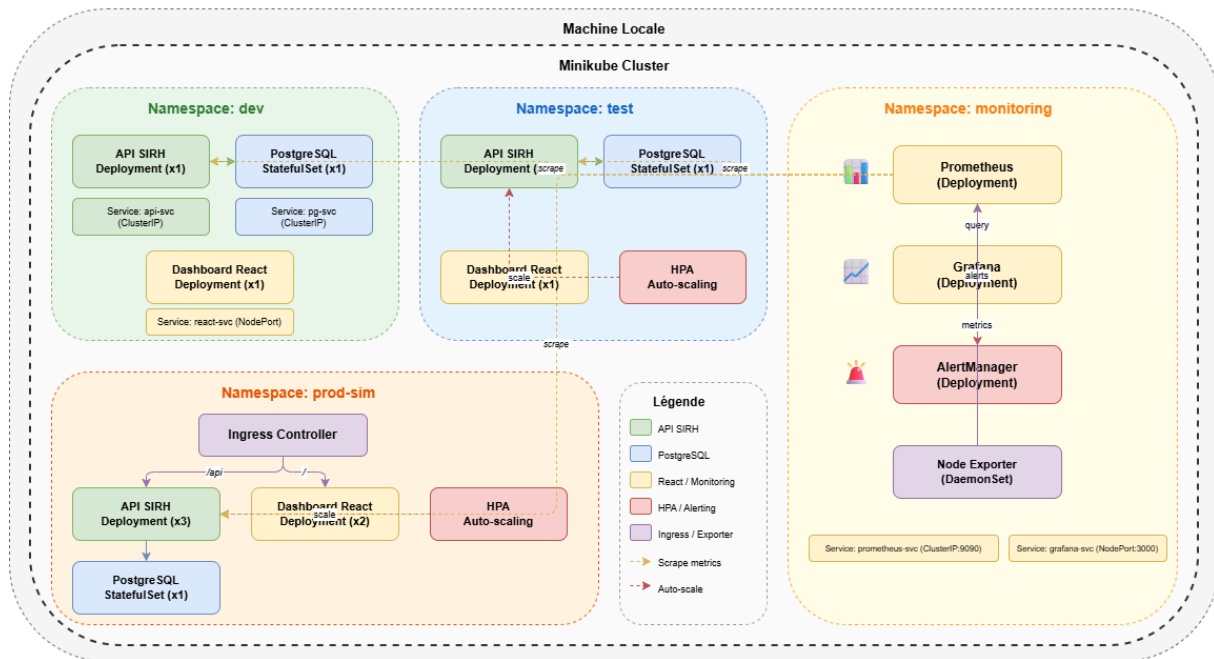
• Déploiement Kubernetes (Minikube)

Ce diagramme illustre l'organisation du cluster Kubernetes local déployé sur Minikube. L'infrastructure est segmentée en quatre namespaces distincts :

- **Dev** : environnement de développement avec des réplicas minimaux
- **Test** : environnement de test intégrant l'auto-scaling (HPA)
- **Prod-Sim** : environnement de production simulé avec un Ingress Controller, des réplicas renforcés et l'auto-scaling activé
- **Monitoring** : namespace dédié à Prometheus, Grafana, AlertManager et Node Exporter.

Cette séparation en namespaces permet de simuler un cycle de vie complet (DEV → TEST → PROD) sur une seule machine locale, tout en respectant les bonnes pratiques d'isolation des environnements.

Figure 3 — Déploiement Kubernetes sur Minikube

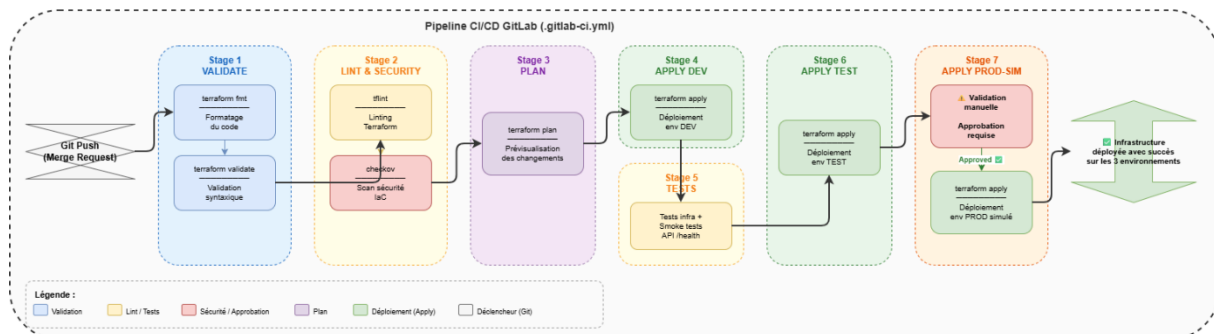


• Pipeline CI/CD GitLab

Ce diagramme décrit le pipeline d'intégration et de déploiement continu mis en place via GitLab CI. Le pipeline est déclenché à chaque merge request ou push sur le dépôt Git et s'exécute en sept étapes séquentielles :

1. **Validate** : formatage (terraform fmt) et validation syntaxique (terraform validate) ;
2. **Lint & Security** : analyse statique (tflint) et scan de sécurité IaC (checkov) ;
3. **Plan** : prévisualisation des changements (terraform plan) ;
4. **Apply DEV** : déploiement automatique sur l'environnement de développement ;
5. **Tests** : tests d'infrastructure et smoke tests applicatifs (endpoint /health) ;
6. **Apply TEST** : déploiement sur l'environnement de test ;
7. **Apply PROD-SIM** : déploiement sur l'environnement de production simulé, conditionné par une validation manuelle (approbation requise)

Figure 4 — Pipeline CI/CD GitLab



Note : “Les diagrammes suivants décrivent respectivement la vue d’ensemble, la séparation en couches, le déploiement sur Minikube (namespaces dev/test/prod-sim) et le pipeline GitLab CI.”

6.4 Observabilité & alerting

L’observabilité est assurée via **Prometheus** (collecte métriques) et **Grafana** (visualisation). Des alertes de base sont définies, par exemple :

- Surcharge CPU sur l’API (seuil + durée),
- Redémarrages anormaux de pods (CrashLoopBackOff),
- Indisponibilité de l’API (endpoint /health).

7. Application SIRH simulée

Une application SIRH volontairement simplifiée est utilisée comme support technique, permettant de générer une charge applicative représentative afin de valider l’infrastructure, l’auto-scaling et le monitoring.

Composition technique (à titre indicatif) :

- **Backend** : API REST RH simulée (endpoints simples, sans logique métier avancée)
- **Frontend** : Dashboard React (visualisation et démonstration)
- **Base de données** : PostgreSQL

NB : L’objectif n’est pas le métier, mais l’infrastructure qui le supporte.

8. Industrialisation & DevOps

- Versionning des modules Terraform via Git
- Séparation des environnements (DEV / TEST / PROD simulés via namespaces Kubernetes)
- Pipeline CI/CD intégrant les étapes suivantes :
 - terraform fmt — Formatage du code
 - terraform validate — Validation syntaxique
 - tflint — Linting des configurations Terraform
 - checkov — Scan de sécurité IaC
 - Tests d’infrastructure automatisés
- Git comme source de vérité (GitOps) pour toute la configuration infrastructure.

9. Sécurité & bonnes pratiques

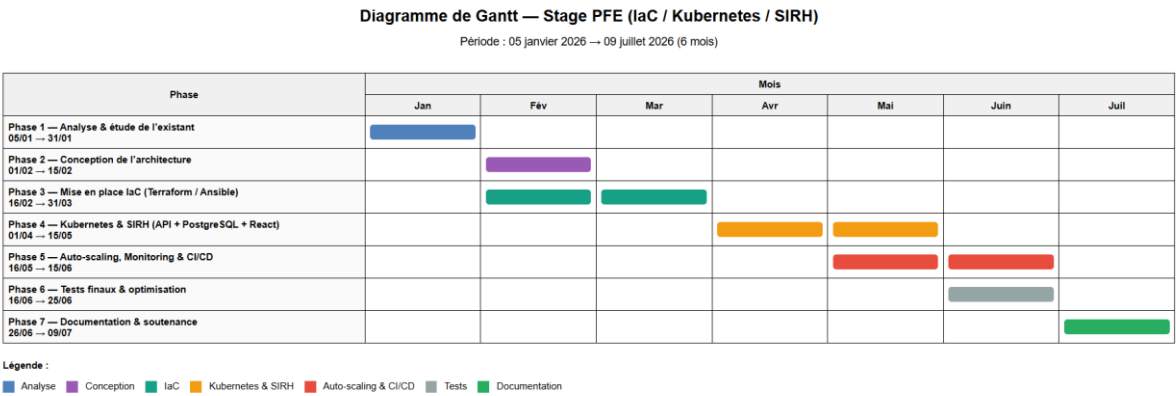
- Isolation réseau : Séparation des réseaux Docker et namespaces Kubernetes
- Gestion des secrets (Variables sécurisées : Kubernetes Secrets, variables d'environnement protégées)
- Scans de sécurité IaC : Checkov, tfint intégrés au pipeline CI/CD
- Contrôle d'accès Kubernetes : RBAC Kubernetes (Role-Based Access Control)

10. Innovation

Un axe d'innovation sera intégré, par exemple :

- **Auto-scaling prédictif simplifié** : Implémentation d'un mécanisme basé sur l'analyse de tendances de métriques (CPU/mémoire) pour anticiper les pics de charge, en complément du HPA réactif de Kubernetes.
- **Policy as Code (sécurité IaC)** : Intégration d'Open Policy Agent (OPA) ou de Checkov pour valider automatiquement la conformité des configurations IaC avant déploiement, garantissant ainsi le respect des règles de sécurité et de gouvernance.

Diagramme de Gantt – Stage PFE (6 mois)



Période totale : 05 janvier → 09 juillet

Phase 1 — Analyse & étude de l'existant : 05 janvier → 31 janvier (4 semaines)

- Prise de connaissance de l'environnement Sopra HR Software
- Analyse des besoins techniques et métiers (SIRH)
- Étude des contraintes (environnement local, sécurité, outillage)
- Benchmark des solutions IaC & Kubernetes local

- Validation du périmètre du projet

Livrable : Cahier des charges validé

Phase 2 — Conception de l'architecture : 01 février → 15 février (2 semaines)

- Conception de l'architecture Cloud-Ready locale
- Définition des couches (Infra, App, Monitoring, CI/CD)
- Choix définitif des outils (Terraform, Ansible, Kubernetes, React)
- Conception des diagrammes d'architecture
- Modélisation des environnements DEV / TEST / PROD (simulés)

Livrables :

- Diagramme d'architecture
- Architecture technique détaillée

Phase 3 — Mise en place de l'Infrastructure as Code : 16 février → 31 mars (6 semaines)

- Mise en place du dépôt Git
- Création des modules Terraform :
 - Réseaux Docker
 - Cluster Kubernetes local
- Mise en place d'Ansible :
 - Provisioning
 - Configuration automatisée
- Versionning et structuration des modules IaC
- Tests de déploiement reproductibles

Livrables : Modules Terraform - Playbooks Ansible

Phase 4 — Déploiement Kubernetes & SIRH simulé : 01 avril → 15 mai (6 semaines)

- Déploiement du cluster Kubernetes local
- Déploiement de l'API SIRH (microservice backend)
- Déploiement de PostgreSQL

- Déploiement du dashboard React
- Configuration des services Kubernetes (Deployments, Services)

Livrables : Cluster Kubernetes opérationnel - Application SIRH simulée fonctionnelle

Phase 5 — Auto-scaling, Monitoring & CI/CD : 16 mai → 15 juin (4 semaines)

- Mise en place de l'auto-scaling (HPA)
- Mise en place de Prometheus & Grafana
- Création des dashboards de monitoring
- Mise en place du pipeline CI/CD :
 - Terraform fmt / validate
 - TFLint / Checkov
- Tests de montée en charge

Livrables : Auto-scaling fonctionnel - Dashboards Grafana - Pipeline CI/CD opérationnel

Phase 6 — Tests finaux & optimisation : 16 juin → 25 juin (1,5 semaine)

- Tests de performance
- Tests de résilience
- Optimisation des configurations
- Corrections finales

Livrable : Plateforme stabilisée

Phase 7 — Documentation & soutenance : 26 juin → 09 juillet (2 semaines)

- Rédaction du rapport PFE
- Documentation technique & utilisateur
- Préparation de la soutenance
- Finalisation des livrables

Livrables finaux : Rapport PFE - Présentation soutenance - Documentation complète